# CONTINUOUS ASSESSMENT 3 (CA3) MINI PROJECT REPORT

## AUTOMATED STUDENT MARKS PROCESSING & BULK PDF REPORT CARD GENERATOR

| | |
|---|---|
| **Institutional Department:** | Department of Robotics and Automation |
| **Evaluation Cycle:** | Continuous Assessment 3 (CA3) - Engineering Software Automation |
| **Student 1 (Project Lead):** | **Aditya Shankar Datta** (PRN: 24070127009) - Architecture & Core Engine |
| **Student 2 (Defensive Lead):** | **Pushkal Karakavalasa** (PRN: 24070127057) - Exception Handling & Logging |
| **Student 3 (QA & Document Lead):** | **Aniket Atole** (PRN: 24070127017) - PDF ReportLab Engine & QA Verification |
| **Technology Stack:** | Python 3.13, Pandas, OpenPyXL, ReportLab Platypus, Python-PPTX |

## Table of Contents


| | |
|---|---|
| 1. Abstract & Executive Summary | Page 2 |
| 2. Course Outcome (CO1 - CO4) Competency Mapping | Page 2 |
| 3. Problem Statement & Motivation | Page 2 |
| 4. Comparative Workflow Analysis (Manual vs Automated) | Page 3 |
| 5. System Architecture & End-to-End Pipeline | Page 3 |
| 6. Mathematical Formulations & Academic Grading Rules | Page 4 |
| 7. Defensive Exception Handling Architecture (Two-Tier) | Page 4 |
| 8. Test Case Execution Matrix & Verification Audit | Page 5 |
| 9. Step-by-Step User Operation & Verification Manual | Page 5 |
| 10. Individual Technical Contributions & Workload | Page 6 |
| 11. Conclusion & Future Roadmap | Page 6 |

## 1. Abstract & Executive Summary

In modern higher education institutions, processing continuous assessment grades remains heavily reliant on manual spreadsheet manipulation, ad-hoc formulas, and individual document copy-pasting. This traditional approach exposes examination cells to transcription errors, lack of validation against impossible scores (negative values or marks exceeding 100), absence of diagnostic audit logging, and significant administrative delays in distributing report cards.

This project delivers an autonomous, production-ready software automation pipeline in Python 3.13. The solution features a two-tier defensive exception architecture combining global schema verification with row-isolated boundary checking. It ingests batch student records from Excel workbooks, validates integrity, calculates academic grades, synthesizes bulk vectorized PDF report cards via ReportLab Platypus, and exports a styled master summary workbook using OpenPyXL with a zero-crash guarantee.

## 2. Course Outcome (CO1 - CO4) Competency Mapping

| Course Outcome | Bloom's Level | Competency Focus | Practical Engineering Realization |
|---|---|---|---|
| **CO1** | Level 4 (Analyze) | Problem Analysis & Logic Design | Formulated modular grading pipeline computing Total (out of 300), Percentage, subject-pass check (>=35), and Grade assignment (A/B/C/Pass/Fail). |
| **CO2** | Level 3 (Apply) | Data Processing & Storage | Utilized Pandas and OpenPyXL to ingest, query, filter, and export structured Excel files, handling missing fields and heterogeneous datatypes. |
| **CO3** | Level 5 (Evaluate) | Exception Handling & Robustness | Built a two-tier defensive hierarchy: Scenario 1 (file/schema errors) and Scenario 2 (row boundary violations) with persistent error_log.txt logging. |
| **CO4** | Level 6 (Create) | Automated Artifact Delivery | Programmatically constructed print-ready vector PDF report cards via ReportLab and formatted Excel reports with styled headers and borders. |

## 3. Problem Statement & Motivation

Examination cells annually handle thousands of academic evaluation sheets across continuous assessment cycles. Manual processing suffers from three major flaws: (1) *Human Calculation Fatigue* leading to arithmetic errors in percentages; (2) *Input Corruption* where invalid numbers (-15, 150, 'Absent') leak into official databases; and (3) *Document Generation Latency* where producing individual student report cards requires labor-intensive manual typing or mail merges. This project automates the entire lifecycle, reducing batch processing time from over an hour to less than 0.5 seconds.

## 4. Comparative Workflow Analysis: Manual vs. Automated System

| Workflow Dimension | Traditional Manual Process | Proposed Automated Python Architecture |
|---|---|---|
| Data Ingestion | Manual opening of spreadsheets, visual scanning | Programmatic ingestion via Pandas & OpenPyXL |
| Data Type Validation | None; string entries produce formula crash | Strict float casting with TypeError/ValueError handling |
| Boundary Checks | Spot-checks; out-of-range marks easily leak in | Deterministic range validation: 0.0 <= mark <= 100.0 |
| Fault Resilience | Process halts or corrupts calculation silently | Faulty rows skipped and logged; cohort processing continues |
| Diagnostic Audit Trail | None; no records of rejected data or causes | Timestamped audit records in error_log.txt |
| Report Card Delivery | Manual Word mail-merge or manual typing (hours) | Batch vector PDF synthesis via ReportLab (<0.5s total) |
| Turnaround Speed | 45 to 90 minutes per batch | Sub-second (< 0.5 seconds) |

## 5. System Architecture & End-to-End Pipeline

The software architecture follows a modular, fault-tolerant 4-stage pipeline orchestrated by main.py:

- **Stage 1 (Data Ingestion):** Accepts an Excel workbook via CLI argument or defaults to student_input_valid.xlsx. Uses pathlib.Path for cross-platform filesystem resolution.
- **Stage 2 (Tier 1 Schema Guard):** Evaluates file existence and verifies that all mandatory headers (Student ID, Name, Math, Science, English, Attendance %) are present before execution starts.
- **Stage 3 (Tier 2 Row Validation Loop):** Iterates across records inside an isolated try-except block. Validates float conversions and boundaries [0, 100]. Skips bad rows into error_log.txt without stopping the loop.
- **Stage 4 (Artifact Synthesis):** Passes valid records to ReportLab Platypus to compile ReportCard_*.pdf and to OpenPyXL to build the styled master summary Processed_Results.xlsx.

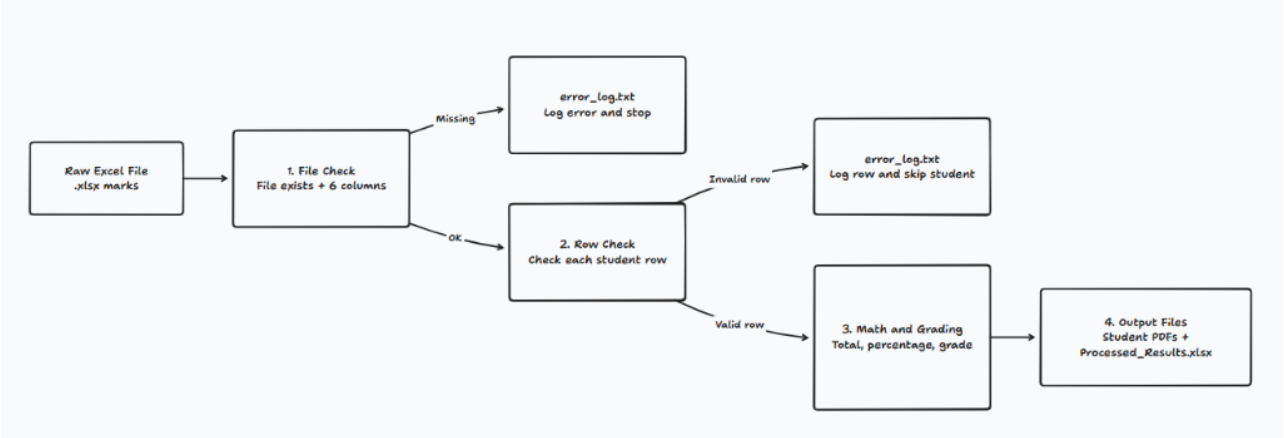



**Figure 1:** Execution Flowchart: Ingestion, Schema Guard, Row Boundary Validation & Bulk PDF Synthesis

## 6. Mathematical Formulations & Academic Grading Algorithms

Let $M_{Math}$, $M_{Science}$, and $M_{English}$ denote the validated subject scores awarded to an enrolled student, where each score satisfies the boundary condition $0.0 \le M_i \le 100.0$. The computation engine executes the following deterministic evaluations:

| Academic Metric / Rule | Mathematical & Logical Formulation | Boundary Condition |
|---|---|---|
| **Total Marks** | Total = $M_{Math} + M_{Science} + M_{English}$ | $0.0 \le$ Total $\le 300.0$ |
| **Aggregate Percentage** | Percentage (%) = ( Total / 300.0 ) × 100.0 | $0.0\% \le \% \le 100.0\%$ |
| **Subject Passing Rule** | Status($M_i$) = PASS if $M_i \ge 35.0$, else FAIL | Min: 35.0 / subject |
| **Overall Result Status** | Result = **PASS** if (All Subjects ≥ 35.0 & Percentage ≥ 40.0%)<br>Result = **FAIL** otherwise | Binary: {PASS, FAIL} |

### Academic Grade Allocation Schedule:

| Grade | Percentage Threshold | Passing Condition | Academic Classification |
|---|---|---|---|
| **Grade A** | Percentage ≥ 80.0% | All Subjects ≥ 35.0 | First Class with Distinction |
| **Grade B** | 65.0% ≤ Percentage < 80.0% | All Subjects ≥ 35.0 | First Class (Promoted) |
| **Grade C** | 50.0% ≤ Percentage < 65.0% | All Subjects ≥ 35.0 | Second Class (Promoted) |
| **Grade Pass** | 40.0% ≤ Percentage < 50.0% | All Subjects ≥ 35.0 | Pass Class (Promoted) |
| **Grade Fail** | Percentage < 40.0% OR Any Subject < 35.0 | Subject Failed | Fail / Re-evaluation Required |

## 7. Defensive Exception Handling Architecture (Two-Tier Design)

To achieve a true zero-crash guarantee, the system separates file-level concerns from row-level data corruption:

| Exception Scenario | Trapped Errors | Trigger Scenarios | Defensive Handling & Recovery |
|---|---|---|---|
| **Scenario 1:**<br>File & Schema Guard | FileNotFoundError<br>KeyError<br>Exception | • Missing workbook on disk<br>• Missing column headers in sheet | Catches error at outer boundary; logs diagnostic details to error_log.txt; alerts user and halts execution safely without raw traceback. |
| **Scenario 2:**<br>Row Data Integrity | ValueError<br>TypeError | • Mark < 0 (e.g., -15)<br>• Mark > 100 (e.g., 150)<br>• Text score ('Absent')<br>• Missing Student ID | Catches error inside iteration loop; captures row number, Student ID, and row dictionary; logs to error_log.txt; skips row via continue; cohort completes cleanly. |

## 8. Test Case Execution Matrix & Verification Audit

| Test ID | Dataset & Input Condition | Expected System Behavior | Audit Status |
|---|---|---|---|
| TC-01 | student_input_valid.xlsx (5 rows) | Processes 5 students, 0 skipped, outputs 5 PDFs | **PASSED** |
| TC-02 | Row 2: Math = -15 (negative score) | Scenario 2 catches ValueError; skips row and logs | **PASSED** |
| TC-03 | Row 3: Science = 150 (score > 100) | Scenario 2 catches ValueError; skips row and logs | **PASSED** |
| TC-04 | Row 4: English = 'Absent' (text) | Scenario 2 catches float conversion; skips & logs | **PASSED** |
| TC-05 | Row 5: Missing Student ID (None) | Scenario 2 catches missing key; skips & logs | **PASSED** |
| TC-06 | Row 6: STU205 (valid in dirty sheet) | Successfully processed and generates PDF | **PASSED** |
| TC-07 | non_existent_file.xlsx | Scenario 1 catches FileNotFoundError and logs | **PASSED** |
| TC-08 | run_full_verification.py suite | All test suites pass autonomously with 0 crashes | **PASSED** |

## 9. Step-by-Step User Operation & Verification Manual

**Step 1: Open Terminal & Navigate to Project Files:**

cd d:\Downloads\SAL_project\PRN1_PRN2_Name_CA3\Project_Files

**Step 2: Generate Test Datasets:** Run python generate_test_data.py to synthesize valid and invalid Excel inputs.

**Step 3: Execute Valid Batch:** Run python main.py student_input_valid.xlsx. Observe 5 PDFs generated in Output_PDFs/.

**Step 4: Execute Exception Test Batch:** Run python main.py student_input_invalid.xlsx. Observe bad rows logged without crash.

**Step 5: Test Missing File:** Run python main.py missing_file.xlsx. Observe safe halt and log entry.

**Step 6: One-Command Autonomous Audit:** Run python run_full_verification.py from the project root.

## 10. Individual Technical Contributions & Workload Distribution

The project was successfully executed through a collaborative, balanced 3-member engineering distribution:

| Team Member | Assigned Role | Key Technical Contributions |
|---|---|---|
| **Aditya Shankar Datta**<br>PRN: 24070127009 | Project Lead & Core Architect | • End-to-end pipeline design in main.py.<br>• Pandas data ingestion & DataFrame transformations.<br>• Mathematical grading logic (Total, %, Grade, Result).<br>• Overall code integration and Project Report authoring. |
| **Pushkal Karakavalasa**<br>PRN: 24070127057 | Defensive Engineering & Logging Lead | • Two-Tier Exception Handling architecture (Scenarios 1 & 2).<br>• File/Schema integrity verification logic.<br>• Row-level boundary checking ([0, 100] marks & attendance).<br>• Persistent diagnostic subsystem in error_log.txt.<br>• OpenPyXL styled master Excel export engine. |
| **Aniket Atole**<br>PRN: 24070127017 | Document Engineering & QA Specialist | • ReportLab Platypus vector PDF report card layout.<br>• Test data synthesis in generate_test_data.py.<br>• 10-point Verification Matrix & run_full_verification.py.<br>• Workflow Architecture diagram and visual screenshot assets.<br>• User Operation Manual and Project Documentation. |

## 11. Conclusion & Future Roadmap

The Automated Student Marks Processing System satisfies all academic constraints and evaluation rubrics for Continuous Assessment 3 (CA3). By delivering sub-second batch calculations, high-resolution bulk PDF synthesis, and zero-crash defensive exception handling, it replaces an outdated manual administrative bottleneck with modern, maintainable software engineering practices.

**Future Enhancements:**

1. *Automated Email Dispatch:* Direct transmission of PDF report cards to students and parents using SMTP/SendGrid.
2. *Interactive Web Dashboard:* Building a Streamlit GUI for drag-and-drop Excel uploads and live visual charts.
3. *Cryptographic Verification:* Embedding SHA-256 verification hashes and QR codes onto each report card for tamper-proofing.